PROJECT SYNOPSIS

SECURE CODE REVIEW ASSISTANT

AI-Powered Debugging and Vulnerability Detection

Submitted by: Mrinmayee Pramod Kulkarni

Roll Number / PRN: 124B2B025

Department: Computer Engineering

College: Pimpri Chinchwad College of Engineering (PCCOE), Pune

Academic Year: 2026–2027

1. Abstract

Software security is an important part of the software development lifecycle. Programming mistakes and insecure coding practices can introduce vulnerabilities that may be difficult to identify through manual inspection alone. The Secure Code Review Assistant is an advanced software application developed by the candidate using Python, Streamlit, and the Google Gemini API to assist developers in reviewing source code and securing applications.

The application provides a web-based interface where users can select a programming-language label, submit source code, and request an automated security review. The analysis engine examines the submitted code and returns potential security vulnerabilities, programming issues, explanations, and recommendations for safer implementation. The interface also presents a summary of the findings and provides a downloadable review report.

The project demonstrates how generative artificial intelligence and static rules can support early-stage debugging and security awareness. It is designed as an interactive assistant for developers to improve code quality before deployment.

Keywords: Secure Code Review, Generative AI, Google Gemini, Python, Streamlit, Vulnerability Detection, Software Security.

2. Introduction

Modern software applications must be developed with attention to correctness, maintainability, and security. Vulnerabilities such as SQL injection, improper input handling, and unsafe coding patterns can expose applications to attacks or unexpected behavior.

Traditional code review often requires developers to inspect source code manually and understand the context in which it executes. This process can be time-consuming, particularly when developers need assistance identifying unfamiliar security weaknesses.

Generative AI provides an additional way to examine code and explain potentially problematic patterns in natural language. The Secure Code Review Assistant incorporates Google's Gemini API to generate code-review feedback through a simple Streamlit interface.
The project focuses on demonstrating an accessible workflow developed by the student for source-code analysis and security recommendations.

3. Problem Statement

Manual code review can require considerable time and specialized knowledge. Beginners may not recognize insecure coding patterns, and simple programming errors can remain unnoticed during development.

There is a need for an accessible tool that accepts source code, identifies potential issues, explains their possible impact, and suggests improvements.

The proposed Secure Code Review Assistant addresses this need by integrating a generative AI model with a browser-based interface for preliminary code-review assistance.

4. Objectives

1. Develop a user-friendly source-code review application.
2. Integrate the Google Gemini API for AI-assisted analysis.
3. Identify potential security vulnerabilities and programming errors.
4. Provide explanations and safer coding recommendations.
5. Present review results through a Streamlit interface.
6. Display a summary of the identified security findings.
7. Provide a downloadable review report.
8. Demonstrate secure handling of API configuration through environment variables.

5. Scope of the Project

The application is designed for educational demonstrations and preliminary code-review assistance.

Its current interface offers Python, C, C++, Java, and JavaScript as language selections. Users can enter or paste source code and request an automated review.

The analysis focuses on potential vulnerabilities, programming mistakes, and code-quality improvements. The results depend on the submitted code and the analysis engine.

The current implementation does not guarantee comprehensive vulnerability detection, compile or formally validate every supported language, or replace dedicated security-analysis tools.

6. Proposed System

The proposed system consists of a Streamlit frontend and a Python analysis module connected to the Gemini API.

The user submits source code through the interface. The analyzer prepares a prompt containing review instructions and the submitted code, then sends the request to the configured Gemini model. The generated response is returned to the application and displayed to the user.

The user can examine the findings and recommendations and use the available report-download functionality.

System Workflow

User submits source code

↓

Selects programming-language context

↓

Streamlit interface (app.py)

↓

Analyzer module (analyzer.py)

↓

Google Gemini API

↓

AI-generated findings and recommendations

↓

Results display and report download

7. Technologies Used

Python: Implements the application's logic and API integration.

Streamlit: Provides the browser-based user interface, code editor, controls, results display, and download option.

Google Gemini API: Generates the code-review analysis.

Google Gen AI Python SDK: Connects the Python application to the Gemini service.

python-dotenv: Loads local environment configuration.

Python virtual environment: Isolates project dependencies.

Notepad and CMD: Used to create and edit source files, install dependencies, and run the application during development.

8. Main Modules

8.1 User Interface Module — app.py
This module displays the application title, programming-language selector, source-code input area, analysis button, identified findings, summary information, and report-download control.

8.2 Code Analysis Module — analyzer.py
This module validates essential input conditions, loads the API configuration, prepares the review prompt, calls the Gemini model, and returns the generated review or an error result.

8.3 Configuration Module — .env
This local configuration file stores the Gemini API key and model name. It must remain private and must not be included in public source-code uploads.

8.4 Sample Input — Sample_Code/Vulnerable_example.py
This file contains sample vulnerable code used to demonstrate the application's review capabilities.

8.5 Dependency File — requirements.txt

This file lists the Python packages required to set up the project environment.

9. Implementation Methodology

The project was implemented in the following stages:

1. Created the project directory and Python virtual environment.
2. Installed the necessary Python packages.
3. Created the configuration file for the Gemini API.
4. Implemented the analyzer module to send source code for review.
5. Developed the Streamlit interface.
6. Configured the application to display the model response and errors.
7. Ran the application locally through CMD.
8. Tested the application using a Python example containing a potential SQL injection weakness.
9. Reviewed the generated findings and recommendations.

10. Demonstration and Observed Results

The application was successfully launched locally and connected to the configured Gemini model.

The test input was:

```
def get_user(user_id):
    query = "SELECT * FROM users WHERE id = " + user_id
    return execute_query(query)
```

During the observed demonstration, the analyzer identified the potential SQL injection risk caused by constructing an SQL query through string concatenation. It recommended parameterized queries as a safer alternative and highlighted other possible issues in the example.

This result demonstrates the intended workflow from source-code submission to automated feedback. It is a functional demonstration, not a quantitative evaluation of vulnerability-detection accuracy.

The report-download feature should be verified in the running application before being presented as a successfully tested result.

11. Advantages

- Simple and accessible browser-based interface.
- Automated identification of potential coding weaknesses.
- Natural-language explanations of findings.
- Suggestions for safer coding practices.
- Separation of interface and analysis logic.
- Configurable AI model through environment variables.
- Potential to support developer learning and security awareness.

12. Limitations

- Analysis results may contain false positives or miss real vulnerabilities.
- Results depend on prompt quality and the submitted code.

- No comprehensive vulnerability benchmark has been established.
- The application does not guarantee complete analysis of large, multi-file projects.
- Generated fixes require review and testing.
- Dedicated static analysis, dynamic testing, and human review remain important.

13. Future Scope

Future development could introduce:

- Integration with static-analysis tools such as Bandit and Semgrep.
- Direct source-file uploads and multi-file project support.
- Structured vulnerability reports with validated severity categories.
- Automated tests for suggested fixes.
- A labelled dataset for measuring detection performance.
- Additional reporting formats, including PDF.
- Improved authentication, usage monitoring, and error handling.
- Sandboxed testing for safely evaluating code changes.

14. Conclusion

The Secure Code Review Assistant demonstrates the application of generative AI to preliminary source-code review. By combining Python, Streamlit, and the Google Gemini API, the system provides an accessible interface for submitting code, reviewing potential vulnerabilities, understanding findings, and receiving safer coding recommendations.

The successful demonstration using a Python example illustrates the potential of the system to support developer productivity and security awareness. However, findings must be verified, and the application should be complemented by established testing and code-review practices before use in security-critical environments.

15. References

1. Google AI for Developers — Gemini API: https://ai.google.dev/gemini-api/docs
2. Google Gen AI Python SDK: https://github.com/googleapis/python-genai
3. Streamlit Documentation: https://docs.streamlit.io/
4. Python Documentation: https://docs.python.org/3/
5. OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
6. python-dotenv: https://pypi.org/project/python-dotenv/

Name and Signature: Mrinmayee Pramod Kulkarni

Date: 15/10/2026